Contact Details

Dr. Mahmud Muhammad
(PhD, MSc, and BSc in Geology)
Email: mahmud.geology@hotmail.com
Website: mahmudm.com

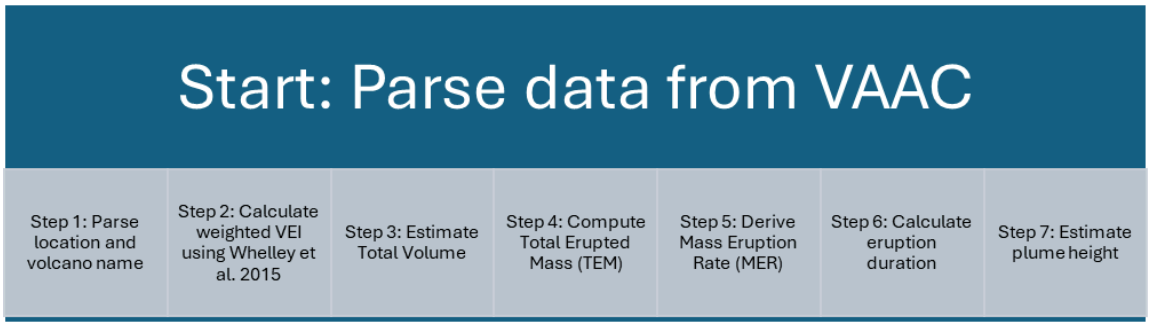
Objective

The goal of this project is to develop a suite of probability density functions (PDFs) that describe Eruption Source Parameter (ESP) inputs required by the NAME model. These estimations are crucial for accurate volcanic ash dispersion modeling.

Key Tasks:

1. **Automated Retrieval and Parsing of Volcanic Advisory Reports (VAA):**
    - Implement a system to automatically retrieve and parse real-time Volcanic Advisory Reports (VAA) from the Tokyo Volcanic Ash Advisory Center (TVAAC).
2. **Annual VEI Probability Parsing:**
    - Utilize the methodology outlined in Whelley et al. (2015) to extract annual Volcanic Explosivity Index (VEI) probabilities for the volcano of concern using data from VAA reports.
3. **Bayesian Sampling for Total Eruption Mass (TEM):**
    - Estimate the range of Total Eruption Mass (TEM) for a known VEI level using Bayesian probability sampling. This estimation will account for a range of tephra particl densities.
4. **Mass Eruption Rate (MER) Estimation:**
    - Calculate the Mass Eruption Rate (MER) based on the derived Total Eruption Mass (TEM).
5. **Eruption Duration Calculation:**
    - Derive the eruption duration by integrating the MER and TEM values.
6. **Plume Height Estimation from MER:**
    - Estimate the volcanic plume height using the Mass Eruption Rate (MER).
7. **MER Estimation from Plume Height:**
    - Estimate the MER using observed Plume Height.
8. **Estimation of Mass fraction of ash particle size below 63 microns:**

- Estimate the Mass fraction of ash particle size below 63 microns using the Mass Eruption Rate (MER).



Parsing of Volcanic Advisory Reports (VAA)

In [30]:
```
# Using Whelley et al., 2015  paper that calculated VEI Level eruption for 750 volc

# first we load Whelley 2015 Volcano Database later will be use to filter Volcanish

import vei

data_loader=vei.LOADDATA()


whelley_data=data_loader.whelley_2015(as_geodataframe=True)
```

In [31]:
```
# import Libraries and classes required to parse VAA reports and filter dataset for

import vep_vaa_text
from vep_vaa_text import VEP_TVAAC_VAAText
scraper = VEP_TVAAC_VAAText()
scraper.fetch_webpage()
tables = scraper.extract_all_tables()
tables
# Perform a search Note: for realtime operation set all parameters to None to retri
search_results = scraper.search(query='SINABUNG', date_time=None, advisory_number='
downloaded_VAA_report, latest_VAA_report , latest_date_data=scraper.download_vaa_te

downloaded_VAA_report.head(2)
```

```
CSV file created: ./vaa_texts_2\vaa_texts.csv
```

Out[31]:

	DTG	VAAC	VOLCANO	VOLCANO CODE	PSN	AREA	SUMMIT ELEV	ADVISC
0	20200810/1635Z	DARWIN	SINABUNG	261080	N0310 E09824	INDONESIA	2460M	2020
1	20200810/1030Z	DARWIN	SINABUNG	261080	N0310 E09824	INDONESIA	2460M	2020

2 rows × 23 columns

In [32]:
```
latest_date_data
```

Out[32]:

	DTG	VAAC	VOLCANO	VOLCANO CODE	PSN	AREA	SUMMIT ELEV	ADVISC
0	20200810/1635Z	DARWIN	SINABUNG	261080	N0310 E09824	INDONESIA	2460M	2020

1 rows × 24 columns

In [33]:
```
# Retrive data from the latest report date
from rich.console import Console
from rich.table import Table

# Initialize the console
console = Console()

# Create a table for a professional and structured output
table = Table(title="Volcanic Activity Observations", show_lines=True, title_style=

# Define table columns
```

```
table.add_column("Parameter", style="dim", width=25)
table.add_column("Value", style="bold white", justify="center")

# Populate the table with data, if available
if 'VOLCANO' in latest_date_data.columns:
    volcano_name = latest_date_data['VOLCANO'].to_list()[0]
    table.add_row("Site of Volcanic Activity", volcano_name)

if 'Altitude_Meters' in latest_date_data.columns:
    observed_ash_altitude_meter = latest_date_data['Altitude_Meters'].to_list()[0]
    table.add_row("Observed Ash Altitude (m ASL)", str(observed_ash_altitude_meter)

if 'Latitude' in latest_date_data.columns:
    latitude = latest_date_data['Latitude'].to_list()[0]
    table.add_row("Latitude", str(latitude))

if 'Longitude' in latest_date_data.columns:
    longitude = latest_date_data['Longitude'].to_list()[0]
    table.add_row("Longitude", str(longitude))



# Call the search_whelley_2015 method on the instance
search_results, vei_range = data_loader.search_whelley_2015(Volcano=volcano_name, m

# View the search results
#print(f"Potential VEI Levels for  Volcano site {volcano_name} is based on Whelley
#print(search_results)

table.add_row(f"Possible VEI Levels for  Volcano  {volcano_name}  based on Whelley

# Render the table
console.print(table)

import pandas as pd
import plotly.graph_objects as go

search_results = pd.DataFrame(search_results)

# Create a table
fig = go.Figure(data=[go.Table(
    header=dict(values=list(search_results.columns),
                fill_color='paleturquoise',
                align='left'),
    cells=dict(values=[search_results[col] for col in search_results.columns],
               fill_color='lavender',
               align='left'))
])

# Add a bold title
fig.update_layout(
    title=dict(
        text=f"Search Results for Whelley et al. 2015 show possible VEI activity fo
        x=0.5,  # Center the title
        font=dict(size=20, family="Arial Black", color="black")  # Use bold font fa
    )
```

```
)

fig.show()
```

```
Top VEI values returned: [2, 3, 4, 5]
```

Volcanic Activity Observations

Parameter	Value
Site of Volcanic Activity	SINABUNG
Observed Ash Altitude (m ASL)	4267.2
Latitude	3.09
Longitude	9.92
Possible VEI Levels for Volcano SINABUNG based on Whelley et al. 2015 paper	[2, 3, 4, 5]

Search Results for Whelley et al. 2015 show possible VEI activity for the volcano SINABUNG

Volcano Number	GVP Volcano Number	Volcano	Other name or Sub-Feature	Latitude	Longitude	Elevation (m)	Classification	Eruption history source	VEI 2	VEI 3	VEI 4	VEI 5	VEI 6	VEI 7	VEI 8
3	261080	SINABUNG	null	3.166666	98.391666	2460	Semi-plugged stratocone	Class	0.022451791	0.002424242	0.001443001	0.0000909091	0	0	0

Step 1: Define Total Eruption Mass (TEM) from VEI

This cell initializes the `VEI_BulkVolume_Mass` class to perform probabilistic calculations of total eruption mass based on known VEI values.

Key Parameters:

- `use_default_densities=True` : Applies default density values for tephra deposits.
- `density_min=800` and `density_max=1700` (kg/m^3): Define the density range for volcanic materials.
- `num_samples=1000` : Specifies the number of Monte Carlo samples for probabilistic calculations.

Outcome:

The `vei_toMass` object enables generating probabilistic TEM values for subsequent calculations.

In [34]:
```
from vei import VEI_BulkVolume_Mass

# Initialize the class
vei_toMass = VEI_BulkVolume_Mass(use_default_densities=False, density_min=800, dens


# Generate probabilistic volumes
vei_toMass.generate_probabilistic_volumes()

# Calculate masses using probabilistic volumes
vei_toMass.calculate_mass(calculate_emperical=True, calculate_probabilistic=True)

vei_toMass.calculate_percentile_bands()



# # Generate summary statistics
vei_toMass.generate_summary_statistics()

# # Visualize the results
vei_toMass.visualize_statistics()

# # Calculate and plot percentile bands

vei_toMass.plot_percentile_bands()

# # Export results
#vei_toMass.export_statistics(filename="summary_statistics.csv")
vei_toMass.export_volumes_and_masses()

vei_toMass.export_data()

#vei_toMass.save_outputs_to_pdf()
```

```
Generating probabilistic volumes...
Probabilistic volumes successfully generated using normal sampling.
Best density sampling strategy: normal
### Volume Percentiles ###
   VEI  emp_Volume_5th  Uncertainty_emp_Volume_5th  Prob_Volume_5th  \
0    0       2300000.0                    230000.0        2300000.0
1    1       2300000.0                    230000.0        2300000.0
2    2       2300000.0                    230000.0        2300000.0
3    3       2300000.0                    230000.0        2300000.0
4    4       2300000.0                    230000.0        2300000.0
5    5       2300000.0                    230000.0        2300000.0
6    6       2300000.0                    230000.0        2300000.0
7    7       2300000.0                    230000.0        2300000.0
8    8       2300000.0                    230000.0        2300000.0

   Uncertainty_Prob_Volume_5th  emp_Volume_25th  Uncertainty_emp_Volume_25th  \
0                     230000.0      100000000.0                   10000000.0
1                     230000.0      100000000.0                   10000000.0
2                     230000.0      100000000.0                   10000000.0
3                     230000.0      100000000.0                   10000000.0
4                     230000.0      100000000.0                   10000000.0
5                     230000.0      100000000.0                   10000000.0
6                     230000.0      100000000.0                   10000000.0
7                     230000.0      100000000.0                   10000000.0
8                     230000.0      100000000.0                   10000000.0

   Prob_Volume_25th  Uncertainty_Prob_Volume_25th  emp_Volume_50th  ...  \
0       100000000.0                    10000000.0     3.000000e+10  ...
1       100000000.0                    10000000.0     3.000000e+10  ...
2       100000000.0                    10000000.0     3.000000e+10  ...
3       100000000.0                    10000000.0     3.000000e+10  ...
4       100000000.0                    10000000.0     3.000000e+10  ...
5       100000000.0                    10000000.0     3.000000e+10  ...
6       100000000.0                    10000000.0     3.000000e+10  ...
7       100000000.0                    10000000.0     3.000000e+10  ...
8       100000000.0                    10000000.0     3.000000e+10  ...

   Prob_Volume_50th  Uncertainty_Prob_Volume_50th  emp_Volume_75th  \
0     3.000000e+10                  3.000000e+09     5.000000e+11
1     3.000000e+10                  3.000000e+09     5.000000e+11
2     3.000000e+10                  3.000000e+09     5.000000e+11
3     3.000000e+10                  3.000000e+09     5.000000e+11
4     3.000000e+10                  3.000000e+09     5.000000e+11
5     3.000000e+10                  3.000000e+09     5.000000e+11
6     3.000000e+10                  3.000000e+09     5.000000e+11
7     3.000000e+10                  3.000000e+09     5.000000e+11
8     3.000000e+10                  3.000000e+09     5.000000e+11

   Uncertainty_emp_Volume_75th  Prob_Volume_75th  \
0                 5.000000e+10      5.000000e+11
1                 5.000000e+10      5.000000e+11
2                 5.000000e+10      5.000000e+11
3                 5.000000e+10      5.000000e+11
4                 5.000000e+10      5.000000e+11
5                 5.000000e+10      5.000000e+11
6                 5.000000e+10      5.000000e+11
```

```
7                 5.000000e+10      5.000000e+11
8                 5.000000e+10      5.000000e+11

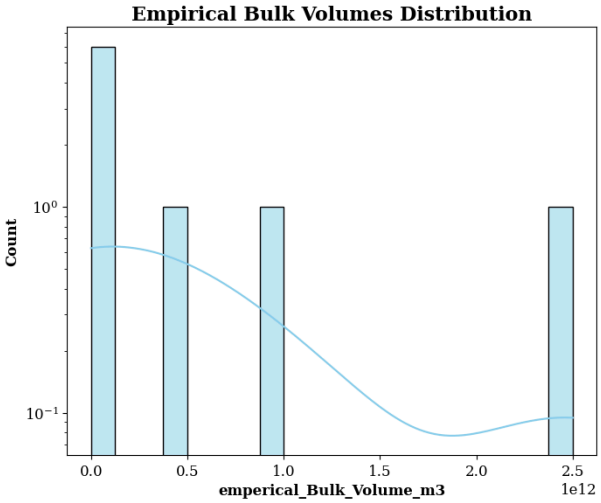
   Uncertainty_Prob_Volume_75th  emp_Volume_95th  Uncertainty_emp_Volume_95th  \
0                  5.000000e+10     1.900000e+12                 1.900000e+11
1                  5.000000e+10     1.900000e+12                 1.900000e+11
2                  5.000000e+10     1.900000e+12                 1.900000e+11
3                  5.000000e+10     1.900000e+12                 1.900000e+11
4                  5.000000e+10     1.900000e+12                 1.900000e+11
5                  5.000000e+10     1.900000e+12                 1.900000e+11
6                  5.000000e+10     1.900000e+12                 1.900000e+11
7                  5.000000e+10     1.900000e+12                 1.900000e+11
8                  5.000000e+10     1.900000e+12                 1.900000e+11

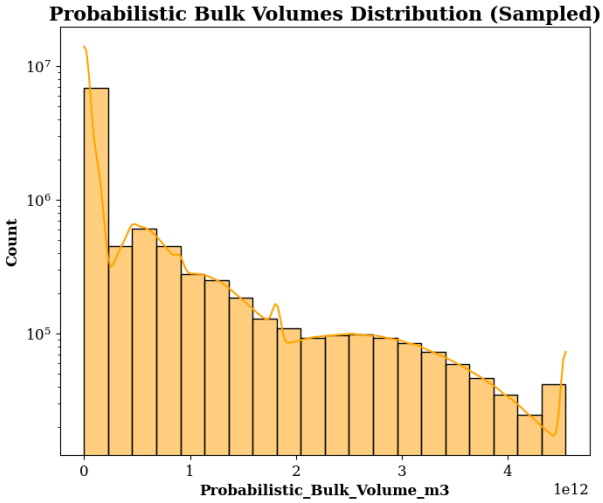
   Prob_Volume_95th  Uncertainty_Prob_Volume_95th
0      1.900000e+12                  1.900000e+11
1      1.900000e+12                  1.900000e+11
2      1.900000e+12                  1.900000e+11
3      1.900000e+12                  1.900000e+11
4      1.900000e+12                  1.900000e+11
5      1.900000e+12                  1.900000e+11
6      1.900000e+12                  1.900000e+11
7      1.900000e+12                  1.900000e+11
8      1.900000e+12                  1.900000e+11

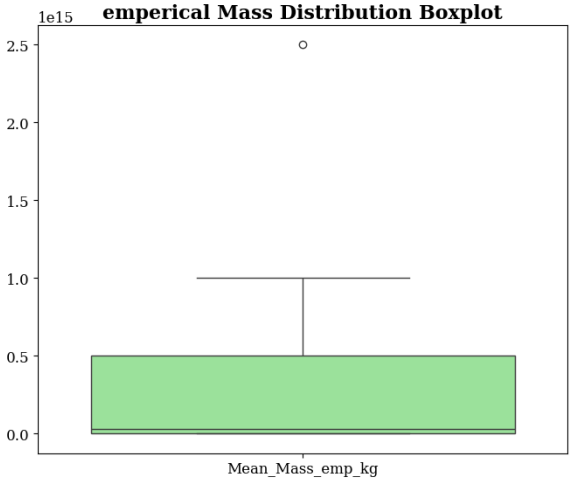
[9 rows x 21 columns]
### Mass Percentiles ###
   VEI      Mass_5th     Mass_25th     Mass_50th     Mass_75th     Mass_95th  \
0    0  3.928194e+08  4.812835e+08  5.427784e+08  6.042238e+08  6.925761e+08
1    1  3.928216e+09  4.813084e+09  5.428078e+09  6.043274e+09  6.927714e+09
2    2  7.857052e+10  9.626567e+10  1.085596e+11  1.208625e+11  1.385496e+11
3    3  2.356816e+12  2.887361e+12  3.256743e+12  3.625434e+12  4.156076e+12
4    4  2.356622e+13  2.887596e+13  3.256434e+13  3.625570e+13  4.155711e+13
5    5  7.856852e+13  9.625030e+13  1.085508e+14  1.208514e+14  1.385393e+14
6    6  3.927524e+14  4.812733e+14  5.428021e+14  6.042862e+14  6.928711e+14
7    7  7.853813e+14  9.623509e+14  1.085388e+15  1.208444e+15  1.385342e+15
8    8  1.963801e+15  2.406527e+15  2.713836e+15  3.021518e+15  3.463603e+15

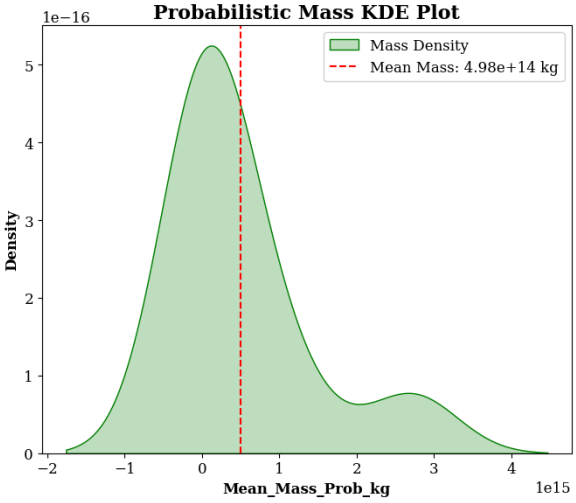
   90%_interpercentile_Mass_Estimate
0                       2.997567e+08
1                       2.999498e+09
2                       5.997906e+10
3                       1.799260e+12
4                       1.799089e+13
5                       5.997083e+13
6                       3.001187e+14
7                       5.999606e+14
8                       1.499803e+15

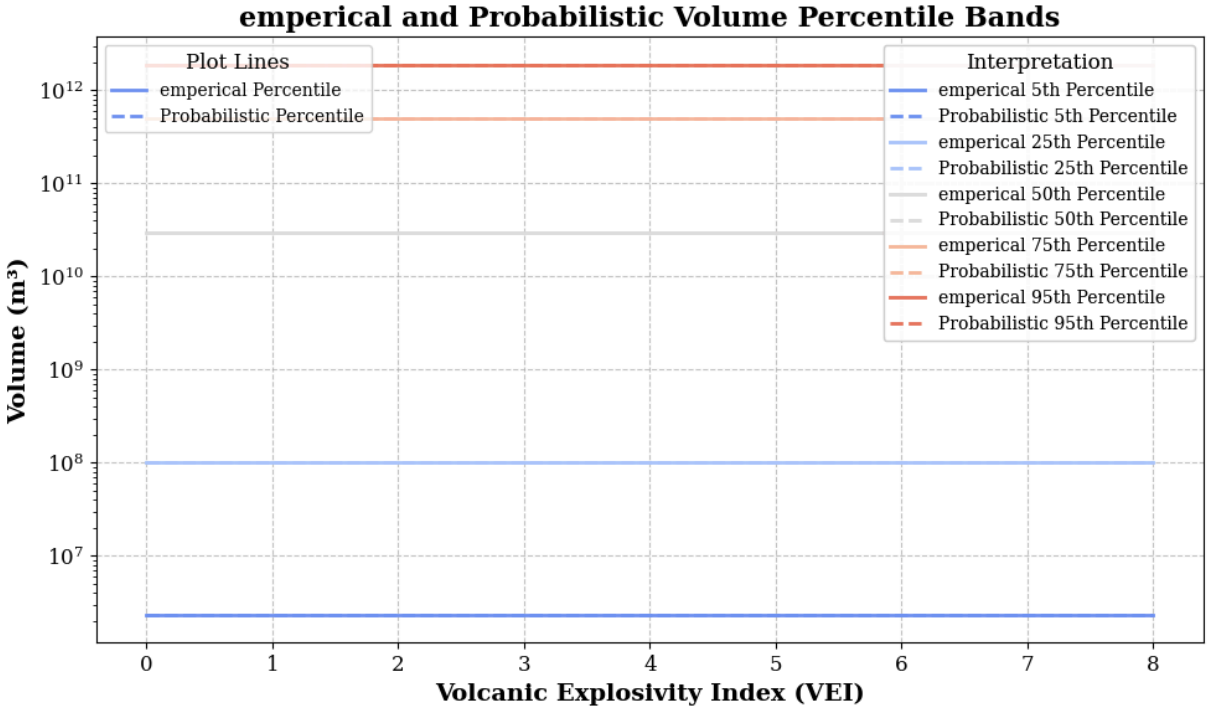
### Generating Summary Statistics Grouped by VEI Level ###
Ensuring VEI is a Series...
Flattening and cleaning VEI column...
Unique VEI values after cleaning: ['0' '1' '2' '3' '4' '5' '6' '7' '8']
Performing groupby operation...
```

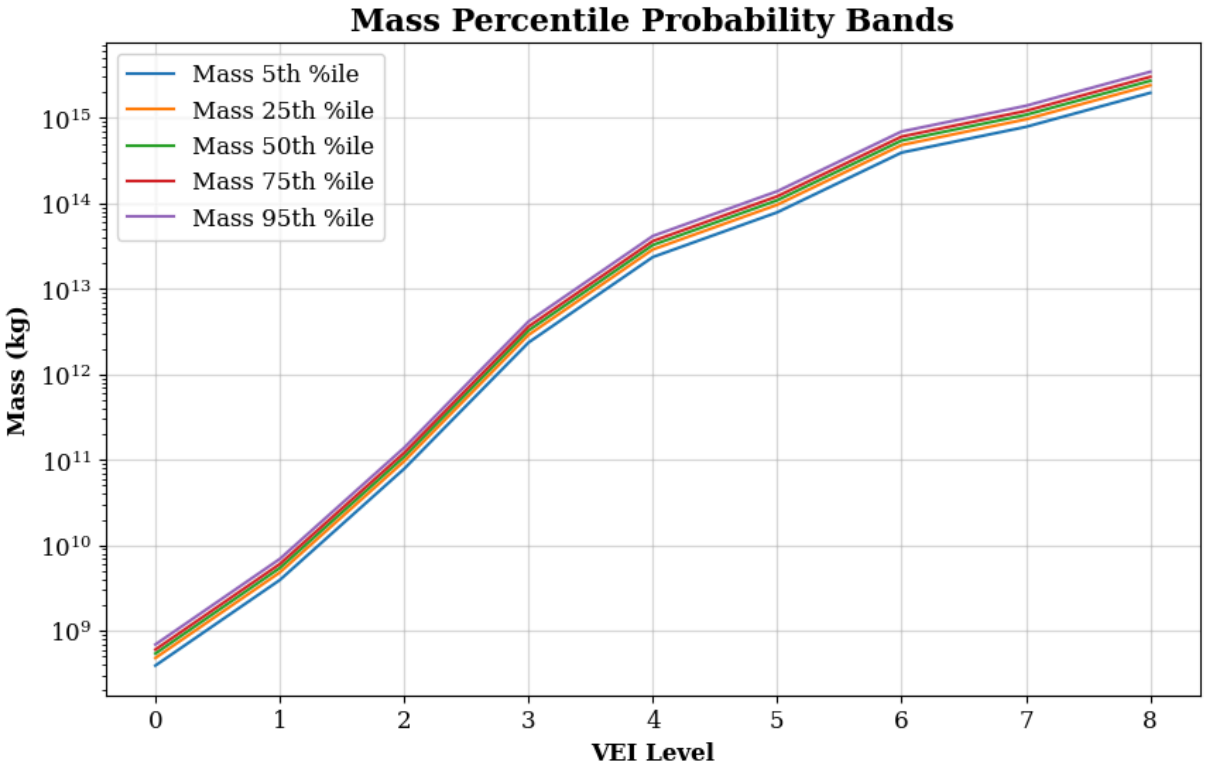

Empirical Bulk Volumes Distribution
Count
emperical_Bulk_Volume_m3
1e12

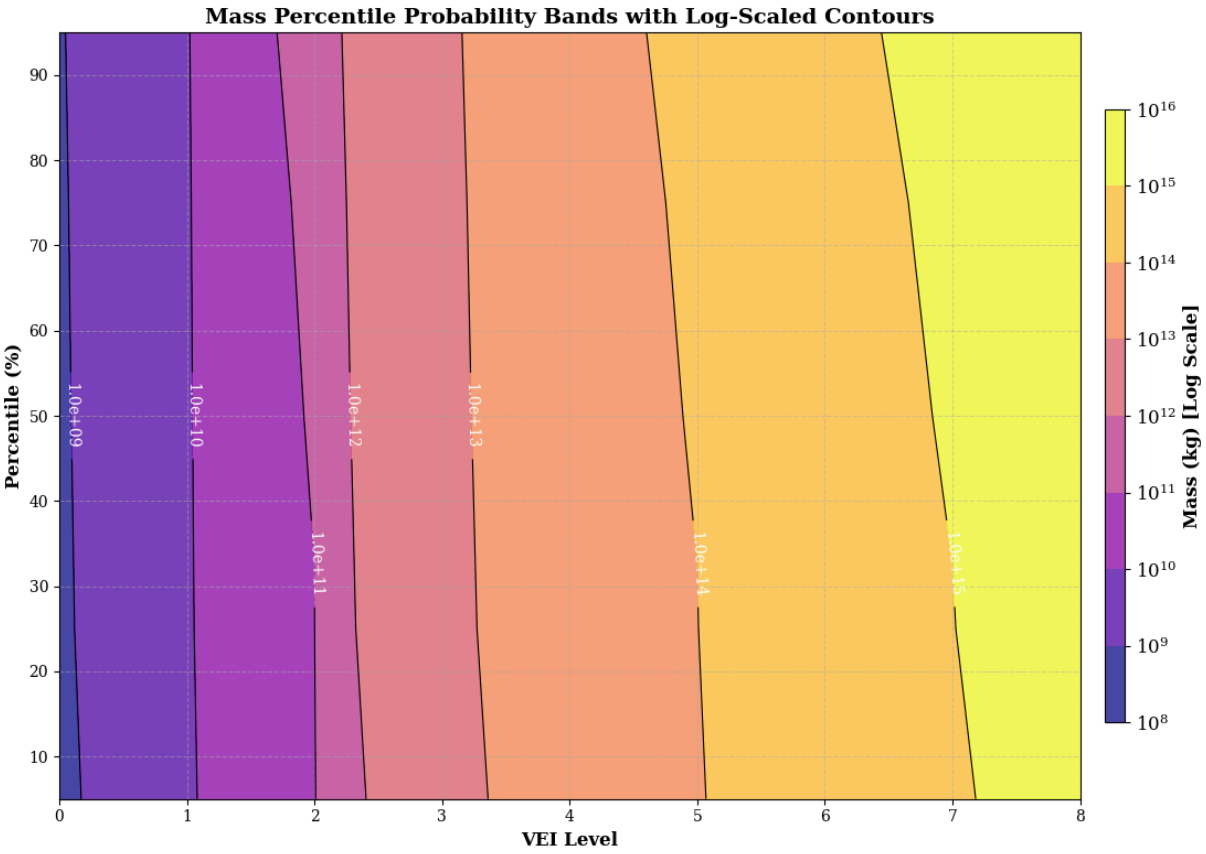

Probabilistic Bulk Volumes Distribution (Sampled)
Count
Probabilistic_Bulk_Volume_m3
1e12


emperical Mass Distribution Boxplot
1e15
Mean_Mass_emp_kg


Probabilistic Mass KDE Plot
1e−16
Mass Density
Mean Mass: 4.98e+14 kg
Density
Mean_Mass_Prob_kg
1e15


emperical and Probabilistic Volume Percentile Bands
Plot Lines
emperical Percentile
Probabilistic Percentile
Interpretation
emperical 5th Percentile
Probabilistic 5th Percentile
emperical 25th Percentile
Probabilistic 25th Percentile
emperical 50th Percentile
Probabilistic 50th Percentile
emperical 75th Percentile
Probabilistic 75th Percentile
emperical 95th Percentile
Probabilistic 95th Percentile
Volume (m³)
Volcanic Explosivity Index (VEI)






```
Volume data successfully exported to volumes_data.csv
Mass data successfully exported to masses_data.csv
Data successfully exported to VEI_MASS_VolumeResults.csv
```

Step 2: Load Volcanic Data for Model Inputs and Validation

This cell loads datasets containing eruption source parameters and metadata.

Data Sources:

1. `Mastin` : Dataset of VEI-related eruption parameters.
2. `Aubrey` : Dataset with parameters for eruption dynamics.
3. `IVESPA` : Integrated Volcanic Eruption Source Parameters Archive for further modeling.

Purpose:

- These datasets provide observational constraints and inputs for modeling eruption parameters.
- Support statistical analysis and correlation studies to validate models.

In [35]:
```
import vei
from vei import  analyze_correlations

data_loader=vei.LOADDATA()


Mastin = data_loader.load_Mastin(as_geodataframe=False)
Aubrey=data_loader.load_Aubry(as_geodataframe=False)
IVESPA=data_loader.load_IVESPA(as_geodataframe=False)
Sparks=data_loader.load_Sparks(as_geodataframe=False)
mastin_a=data_loader.load_Mastin_a(as_geodataframe=True)

# analyze_correlations(Mastin, 'Mastin', threshold=0.7)

# analyze_correlations(Aubrey, 'Aubrey', threshold=0.7)

# analyze_correlations(Sparks, 'Sparks', threshold=0.7)

# analyze_correlations(IVESPA, 'IVESPA', threshold=0.7)

#analyze_correlations(mastin_a, 'IVESPA', threshold=0.9)

import pandas as pd

Combined_datasets= pd.concat([ Aubrey, IVESPA, mastin_a], axis=0, join='inner')

aubrey_datacombined= pd.concat([ Aubrey, IVESPA], axis=0, join='inner')
```

Step 3: Estimate Mass Eruption Rate (MER)

This cell prepares TEM data for use with the `MERPredictor` class.

Inputs:

- `tem_values` : Extracted mean probabilistic TEM values (`Mean_Mass_Prob_kg`) generated by the VEI model.

Purpose:

- `tem_values` serve as input to estimate MER, an essential eruption source parameter.
- MER will be used in subsequent steps to calculate eruption duration and plume rise.

In [36]:
```
import pandas as pd

data=vei_toMass.data

# Drop first row (VEI zero) because VEI zero is None Explosive

#data = data.drop(index=data.index[0]).reset_index(drop=True)


# Ensuring the VEI column is numeric
data['VEI'] = pd.to_numeric(data['VEI'], errors='coerce')

# Filtering the dataframe based on VEI column and list
data = data[data['VEI'].isin(vei_range)].reset_index(drop=True)

data
```

Out[36]:

	VEI	Bulk_Volume_km3	emperical_Bulk_Volume_m3	Probabilistic_Bulk_Volume_m3	Mean
0	2	0.1	1.000000e+08	1.000000e+08	
1	3	3.0	3.000000e+09	3.000000e+09	
2	4	30.0	3.000000e+10	3.000000e+10	
3	5	100.0	1.000000e+11	1.000000e+11	

4 rows × 34 columns

In [37]:
```
data.columns
```

Out[37]:
```
Index(['VEI', 'Bulk_Volume_km3', 'emperical_Bulk_Volume_m3',
       'Probabilistic_Bulk_Volume_m3', 'Mean_Mass_emp_kg', 'Std_Mass_emp_kg',
       'Mean_Mass_Prob_kg', 'Std_Mass_Prob_kg', 'emp_Volume_5th',
       'Uncertainty_emp_Volume_5th', 'Prob_Volume_5th',
       'Uncertainty_Prob_Volume_5th', 'emp_Volume_25th',
       'Uncertainty_emp_Volume_25th', 'Prob_Volume_25th',
       'Uncertainty_Prob_Volume_25th', 'emp_Volume_50th',
       'Uncertainty_emp_Volume_50th', 'Prob_Volume_50th',
       'Uncertainty_Prob_Volume_50th', 'emp_Volume_75th',
       'Uncertainty_emp_Volume_75th', 'Prob_Volume_75th',
       'Uncertainty_Prob_Volume_75th', 'emp_Volume_95th',
       'Uncertainty_emp_Volume_95th', 'Prob_Volume_95th',
       'Uncertainty_Prob_Volume_95th', 'Mass_5th', 'Mass_25th', 'Mass_50th',
       'Mass_75th', 'Mass_95th', '90%_interpercentile_Mass_Estimate'],
      dtype='object')
```

In [38]:
```
from mer_predict import MERPredictor

tem_values=data['90%_interpercentile_Mass_Estimate'].to_list()



#tem_values=[1.998468e+11 , 3.585472e+14 ]

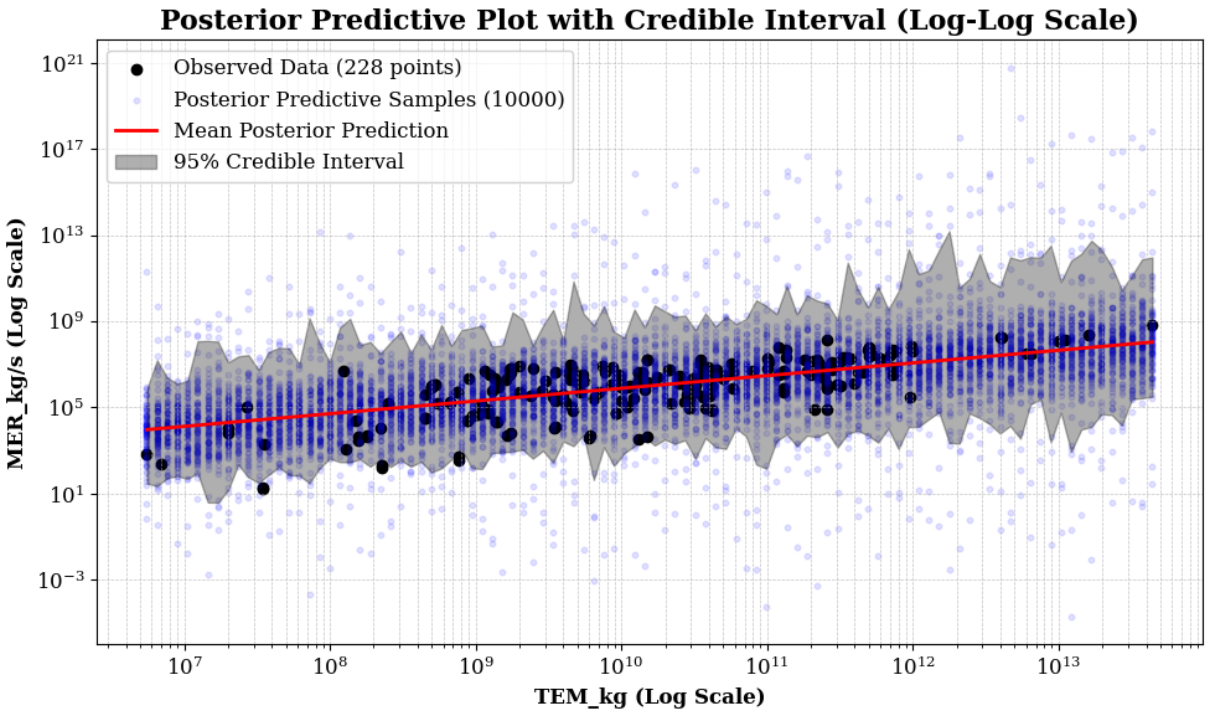


model = MERPredictor(aubrey_datacombined)

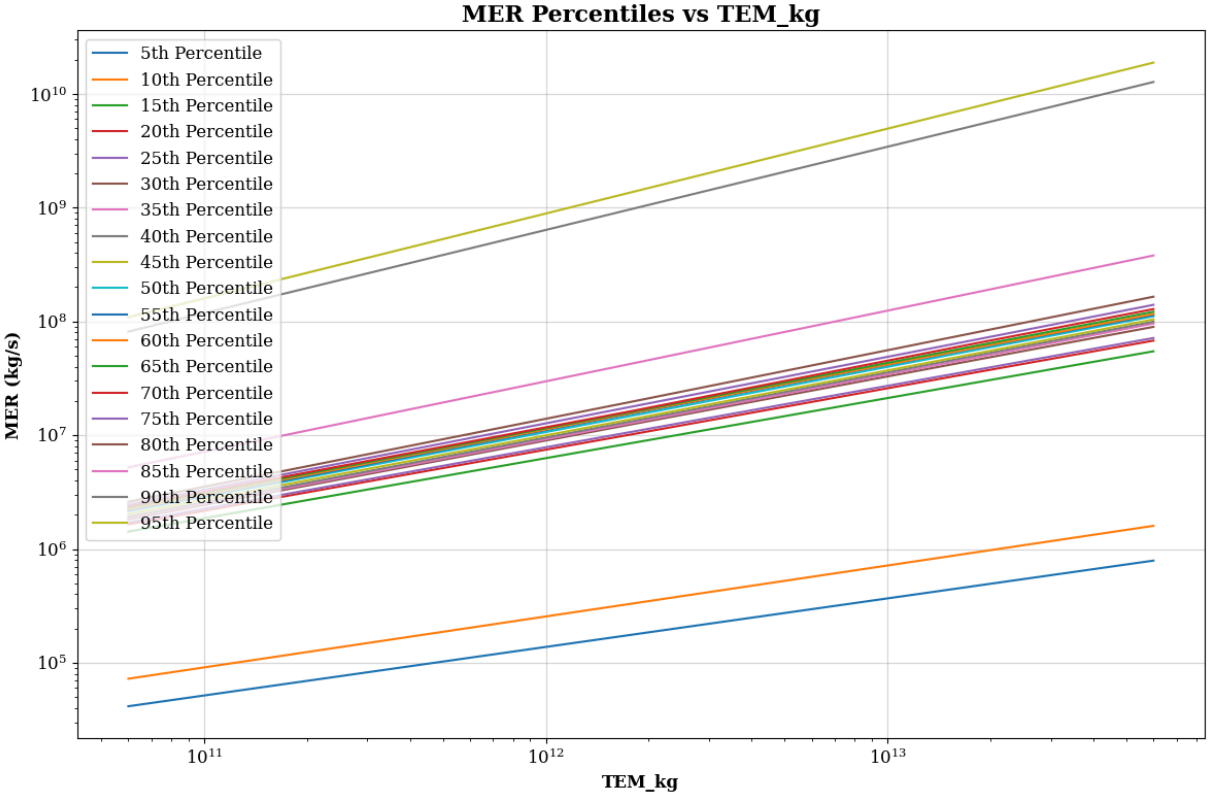
percentiles=list(range(5, 96, 5))

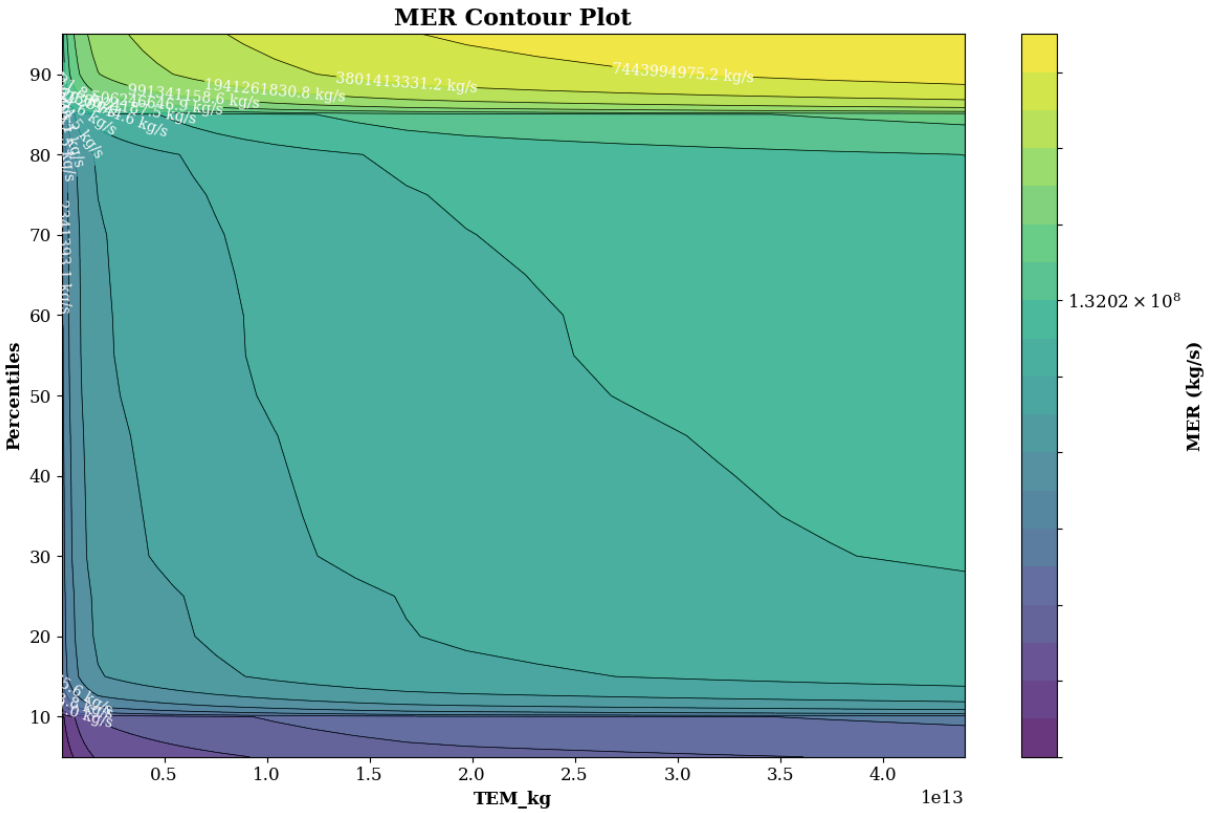
model.plot_posterior_predictive()
mer_results=model.predict_mer(tem_values, percentiles=percentiles)
#model.plot_percentiles_vs_tem()
# durations = model.convert_percentiles_to_duration(tem_values)
# print(durations)
```

```
Selected model: linear with BIC = 627.2178344726562
```



Predictions saved to predicted_mer.csv





Step 4: Correlate VEI with MER Results

This cell adds the VEI values from the processed data into the `mer_results` DataFrame.

Purpose:

- Ensures that MER results are directly associated with their corresponding VEI values.
- Facilitates traceability and cross-comparison during analysis.

In [39]:
```
mer_results["VEI"]=data['VEI']
```

Step 5: Review Mass Eruption Rate (MER) Results

This cell displays the `mer_results` DataFrame, which includes the calculated MER values and associated VEI data.

Purpose:

- Provides an overview of the MER estimates to verify accuracy and consistency.

In [40]:
```
mer_results.columns
```

Out[40]:
```
Index(['TEM_kg', 'MERPercentile_5', 'MERPercentile_10', 'MERPercentile_15',
       'MERPercentile_20', 'MERPercentile_25', 'MERPercentile_30',
       'MERPercentile_35', 'MERPercentile_40', 'MERPercentile_45',
       'MERPercentile_50', 'MERPercentile_55', 'MERPercentile_60',
       'MERPercentile_65', 'MERPercentile_70', 'MERPercentile_75',
       'MERPercentile_80', 'MERPercentile_85', 'MERPercentile_90',
       'MERPercentile_95', '90%_interpercentile_MERrange_kg/s',
       'Duration_hr_P5', 'Duration_hr_P10', 'Duration_hr_P15',
       'Duration_hr_P20', 'Duration_hr_P25', 'Duration_hr_P30',
       'Duration_hr_P35', 'Duration_hr_P40', 'Duration_hr_P45',
       'Duration_hr_P50', 'Duration_hr_P55', 'Duration_hr_P60',
       'Duration_hr_P65', 'Duration_hr_P70', 'Duration_hr_P75',
       'Duration_hr_P80', 'Duration_hr_P85', 'Duration_hr_P90',
       'Duration_hr_P95', 'Duration_hr_90%_interpercentile', 'VEI'],
      dtype='object')
```

Step 6: Select Central MER Values for Plume Height Prediction

This cell extracts the 50th percentile (median) MER values from the `mer_results` DataFrame.

Purpose:

- Focuses on the central tendency of the MER distribution, minimizing the influence of outliers.
- The Best MER values (`mer_list`) are determined using the range between the (`95th and 5th percentiles`) and are used as input for plume height predictions..

In [41]:
```
mer_list=mer_results['MERPercentile_95'].to_list()
vei_list=mer_results.VEI.to_list()
vei_list
```

Out[41]:
```
[2, 3, 4, 5]
```

Step 7: Estimate Plume Rise Based on MER

This cell initializes the `PlumeHeightPredictor` class and calculates plume heights using the median MER values.

Methodology:

- Utilizes the `IVESPA` dataset for plume height predictions with uncertainty quantification.
- Outputs plume height estimates based on the MER values (`mer_list`).

Purpose:

- Plume rise estimation is critical for assessing the environmental and aviation impacts of volcanic eruptions.

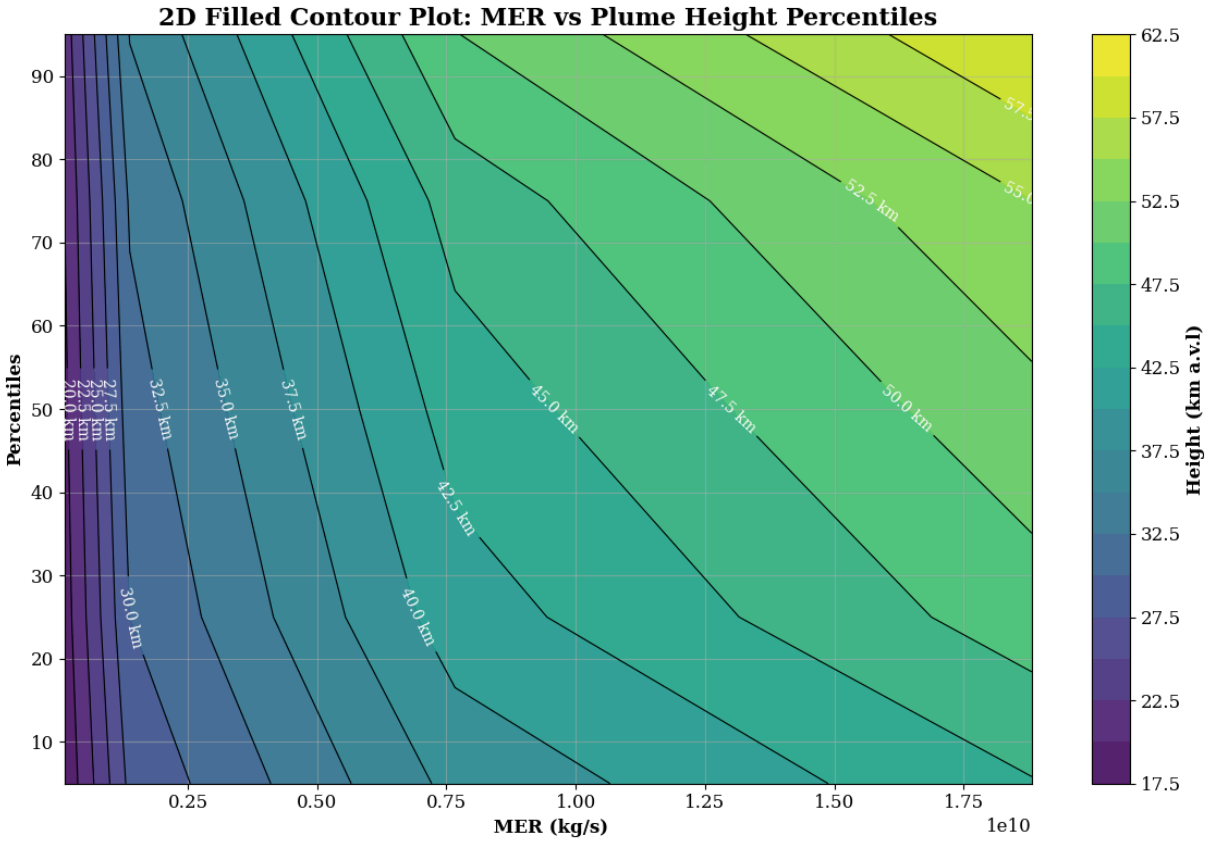
In [42]:
```
from plume_rise_predict import PlumeHeightPredictor

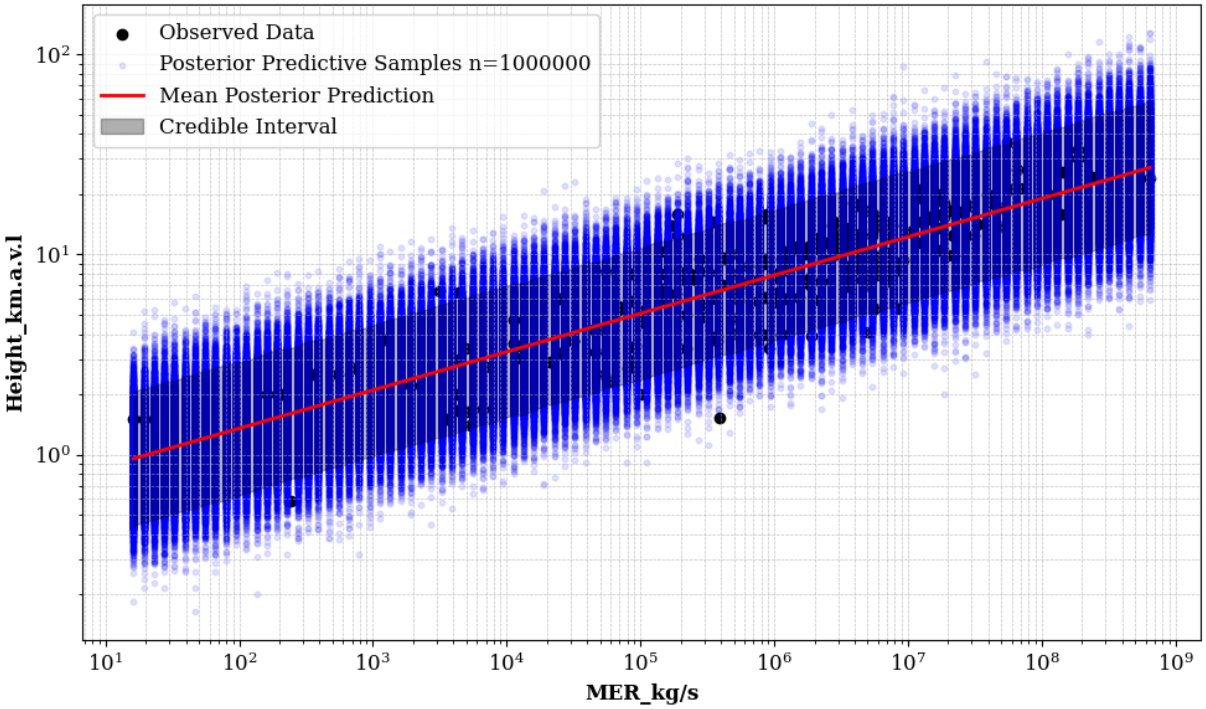
# Initialize the PlumeHeightPredictor
predictor = PlumeHeightPredictor(aubrey_datacombined)

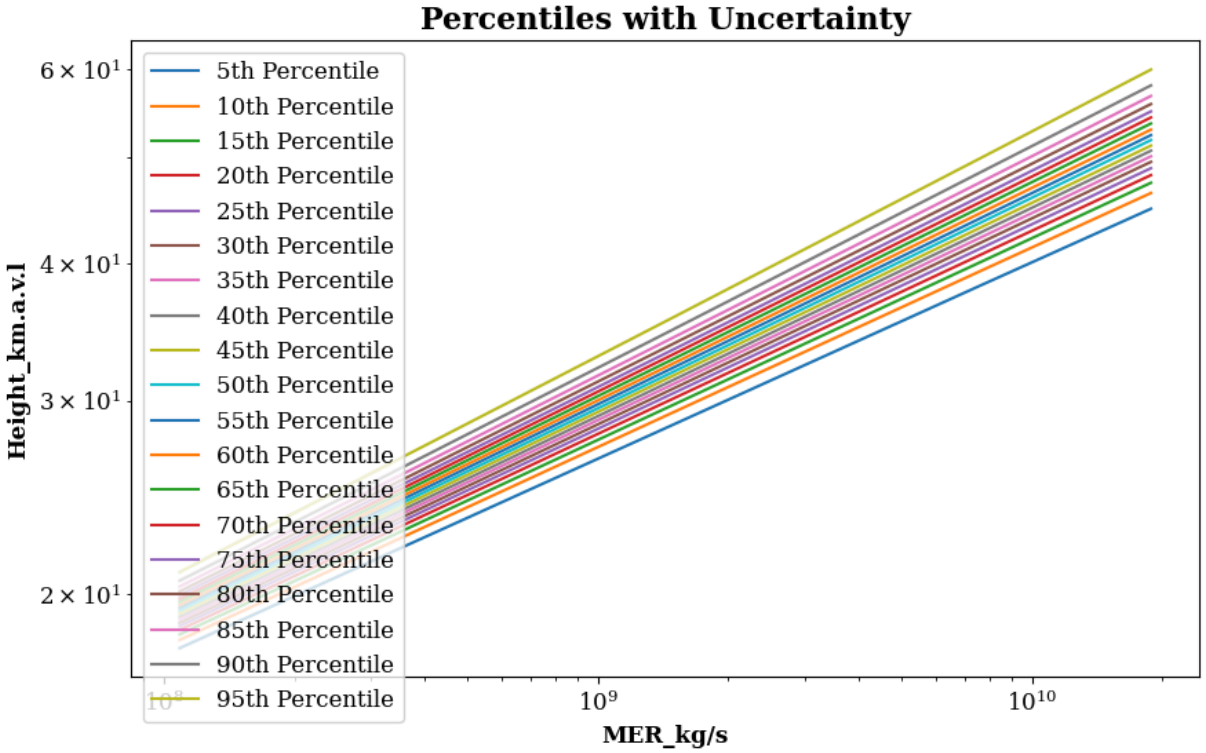
heights=predictor.predict_height_with_uncertainty(mer_results['MERPercentile_95'].t


heights
```

```
Predictions with uncertainty saved to predicted_height_with_uncertainty.csv
```




Observed Data
Posterior Predictive Samples n=1000000
Mean Posterior Prediction
Credible Interval
Height_km.a.v.l
MER_kg/s


Percentiles with Uncertainty
5th Percentile
10th Percentile
15th Percentile
20th Percentile
25th Percentile
30th Percentile
35th Percentile
40th Percentile
45th Percentile
50th Percentile
55th Percentile
60th Percentile
65th Percentile
70th Percentile
75th Percentile
80th Percentile
85th Percentile
90th Percentile
95th Percentile
Height_km.a.v.l
MER_kg/s

Out[42]:

	MER_kg/s	HeightPercentile_5	HeightPercentile_10	HeightPercentile_15	HeightPercer
0	1.089304e+08	17.869545	18.177558	18.389967	18
1	1.376456e+09	28.119691	28.846698	29.298892	29
2	7.666528e+09	38.223290	39.401119	40.142852	40
3	1.882030e+10	44.833434	46.336032	47.339993	48

4 rows × 21 columns

Step 8: Review Plume Height Estimates

This cell displays the predicted plume heights.

Purpose:

- Offers an immediate review of the plume height results for quality control and interpretation.

In [43]:
```
mer_results.columns
```

Out[43]:
```
Index(['TEM_kg', 'MERPercentile_5', 'MERPercentile_10', 'MERPercentile_15',
       'MERPercentile_20', 'MERPercentile_25', 'MERPercentile_30',
       'MERPercentile_35', 'MERPercentile_40', 'MERPercentile_45',
       'MERPercentile_50', 'MERPercentile_55', 'MERPercentile_60',
       'MERPercentile_65', 'MERPercentile_70', 'MERPercentile_75',
       'MERPercentile_80', 'MERPercentile_85', 'MERPercentile_90',
       'MERPercentile_95', '90%_interpercentile_MERrange_kg/s',
       'Duration_hr_P5', 'Duration_hr_P10', 'Duration_hr_P15',
       'Duration_hr_P20', 'Duration_hr_P25', 'Duration_hr_P30',
       'Duration_hr_P35', 'Duration_hr_P40', 'Duration_hr_P45',
       'Duration_hr_P50', 'Duration_hr_P55', 'Duration_hr_P60',
       'Duration_hr_P65', 'Duration_hr_P70', 'Duration_hr_P75',
       'Duration_hr_P80', 'Duration_hr_P85', 'Duration_hr_P90',
       'Duration_hr_P95', 'Duration_hr_90%_interpercentile', 'VEI'],
      dtype='object')
```

In [44]:
```
heights['VEI']=mer_results['VEI']
# List of columns to be assigned
columns_to_assign = ['Duration_hr_P5', 'Duration_hr_P25', 'Duration_hr_P50', 'Durat

# Assign values from mer_results to heights for these columns
heights[columns_to_assign] = mer_results[columns_to_assign]



heights.to_csv('Final_prediction_results.csv')
```

```
heights
```

Out[44]:

	MER_kg/s	HeightPercentile_5	HeightPercentile_10	HeightPercentile_15	HeightPercer
0	1.089304e+08	17.869545	18.177558	18.389967	18
1	1.376456e+09	28.119691	28.846698	29.298892	29
2	7.666528e+09	38.223290	39.401119	40.142852	40
3	1.882030e+10	44.833434	46.336032	47.339993	48

4 rows × 28 columns

In [45]:
```
# Add a bold title using Styler
styled_table = heights.style.set_caption("<b>VEI and Related Data</b>")
styled_table
```

Out[45]:

	MER_kg/s	HeightPercentile_5	HeightPercentile_10	HeightPercentile_15	Heigh
0	108930427.805642	17.869545	18.177558	18.389967	
1	1376455883.512997	28.119691	28.846698	29.298892	
2	7666527950.317772	38.223290	39.401119	40.142852	
3	18820298071.726933	44.833434	46.336032	47.339993	

In [46]:
```
from rich.console import Console
from rich.table import Table

# Ensure VEI is first
if "VEI" not in heights.columns:
    raise KeyError("The 'VEI' column is missing from the DataFrame.")

cols = ["VEI"] + [col for col in heights.columns if col != "VEI"]
heights = heights[cols]

# Rich Console for better formatting
console = Console()

# Function to print DataFrame in chunks of six columns, keeping VEI first
def print_six_columns_with_vei_first(df):
    num_cols = len(df.columns)
    for i in range(1, num_cols, 3):  # 5 because VEI is fixed
        # Slice five columns at a time, after the first column
        cols = ["VEI"] + list(df.columns[i:i+3])

        # Create a table for this chunk
        table = Table(title="VEI and Related Data")
        for col in cols:
```

```
            table.add_column(col, justify="center")

        # Add rows to the table
        for _, row in df[cols].iterrows():
            table.add_row(*[str(cell) for cell in row])

        # Print the table
        console.print(table)

# Display the DataFrame
print_six_columns_with_vei_first(heights)
```

VEI and Related Data

VEI	MER_kg/s	HeightPercentile_5	HeightPercentile_10
2.0	108930427.80564217	17.86954462828855	18.177557721163666
3.0	1376455883.5129974	28.119691210959836	28.846697557089463
4.0	7666527950.317772	38.223290227926775	39.401118901609976
5.0	18820298071.726933	44.83343394808933	46.33603214119359

VEI and Related Data

VEI	HeightPercentile_15	HeightPercentile_20	HeightPercentile_25
2.0	18.389967148231104	18.564386195170545	18.71256839802606
3.0	29.29889211899581	29.68130030060635	30.00999744008093
4.0	40.14285214388028	40.7591896939232	41.30353640787079
5.0	47.33999319471453	48.104108057447235	48.80667244719895

VEI and Related Data

VEI	HeightPercentile_30	HeightPercentile_35	HeightPercentile_40
2.0	18.841454320496737	18.96453776182292	19.083594429693928
3.0	30.302604720369885	30.585132367816247	30.84727997787446
4.0	41.80180906811698	42.2578433550513	42.71147452109582
5.0	49.45770897425214	50.03584750891737	50.62853112326255

VEI and Related Data

VEI	HeightPercentile_45	HeightPercentile_50	HeightPercentile_55
2.0	19.204301572107084	19.331435430798102	19.448101288946695
3.0	31.1280937226267	31.376833969318113	31.6490347699124
4.0	43.15149962460186	43.596145448704746	44.01380450015622
5.0	51.17177781642558	51.769365989289234	52.307060641657024

VEI and Related Data

VEI	HeightPercentile_60	HeightPercentile_65	HeightPercentile_70
2.0	19.56007146725412	19.691315467071444	19.81670609241181
3.0	31.92491476100049	32.21840605596899	32.536092969893076
4.0	44.48883270693787	45.010477547614876	45.52079292188223
5.0	52.90994680989457	53.59343196471567	54.27578975824699

VEI and Related Data

VEI	HeightPercentile_75	HeightPercentile_80	HeightPercentile_85
2.0	19.962671121722288	20.117338633150386	20.326254109844836
3.0	32.861484151728	33.25252758541762	33.68485497349372
4.0	46.064803316998606	46.73368188264804	47.44936858646855
5.0	54.97206607116358	55.825630626906666	56.7744590256307

VEI and Related Data

VEI	HeightPercentile_90	HeightPercentile_95	90%_interpercentile_Heigh
2.0	20.575431691249847	20.95321380180236	3.08366917351380
3.0	34.25975292919227	35.12767578863031	7.00798457767047
4.0	48.45120428015477	49.911250950970995	11.6879607230442
5.0	58.044681829402705	60.00297375376808	15.16953980567875

VEI and Related Data

VEI	Duration_hr_P5	Duration_hr_P25	Duration_hr_P50
2.0	401.225667609912	9.733775317736	7.76150738560531
3.0	2820.555778204933	46.41822109197159	33.5328698060033
4.0	10560.450044394614	133.8743816198427	89.36486034481432
5.0	21061.359919666374	232.93574228926144	150.11148091124025

VEI and Related Data

VEI	Duration_hr_P75	Duration_hr_P95	Duration_hr_90%_interpercen
2.0	6.783100726101502	0.1529494671378146	0.1530077945643373
3.0	27.8731403695691	0.3631024084300116	0.3631491582000323
4.0	72.57141826496915	0.6518555806407531	0.6518958196387353
5.0	119.07381057465625	0.8851381425542187	0.8851753434976701

Refining Mass Eruption Rate (MER) Estimates Using Observed Plume Height

When plume height is observed through satellite imagery or infrasound data, it provides an opportunity to enhance previously predicted MER values. By leveraging this direct observation of plume height, we can derive a more accurate estimate of the MER, aligning it closely with the actual eruption dynamics.

In [47]:
```
from mer_from_height import MERPredictorFromHeight

predictor = MERPredictorFromHeight(Combined_datasets)
# Predict MER for specific heights
# Save predictions to a CSV
Observed_plume_height=[5,8]
predictions=predictor.predict_MER_with_uncertainty(Observed_plume_height, output_fi

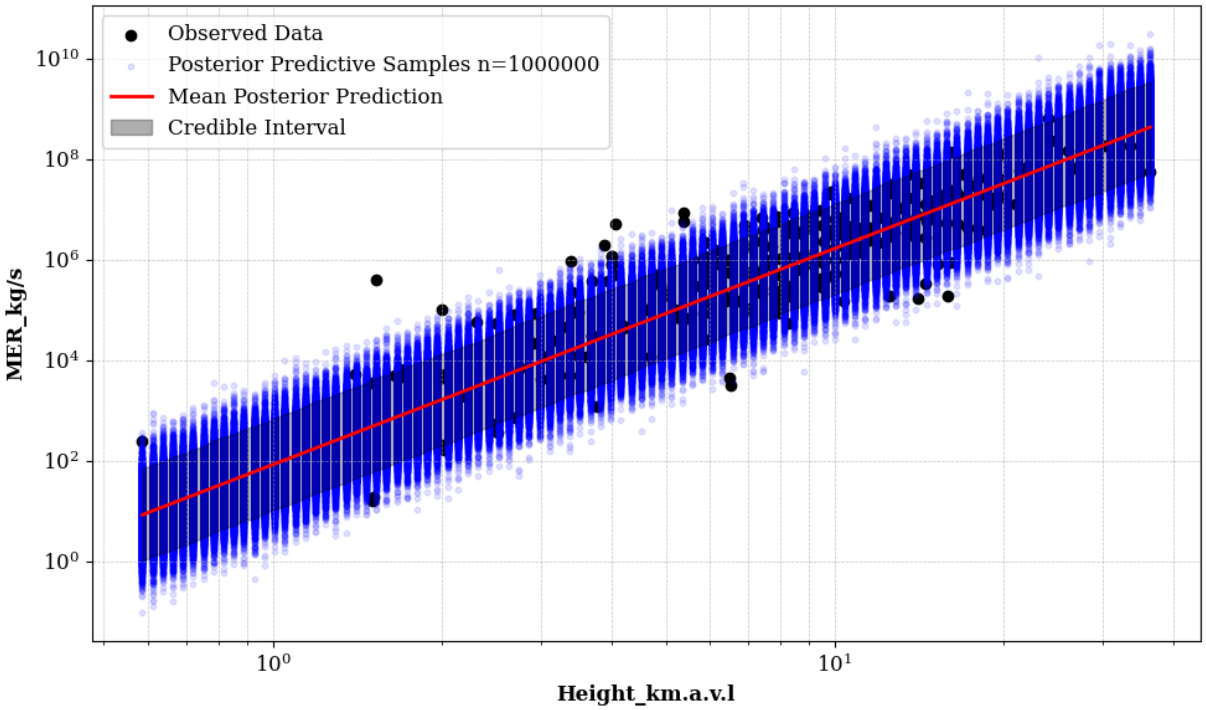

# Visualize the posterior predictive distribution
predictor.plot_posterior_predictive()

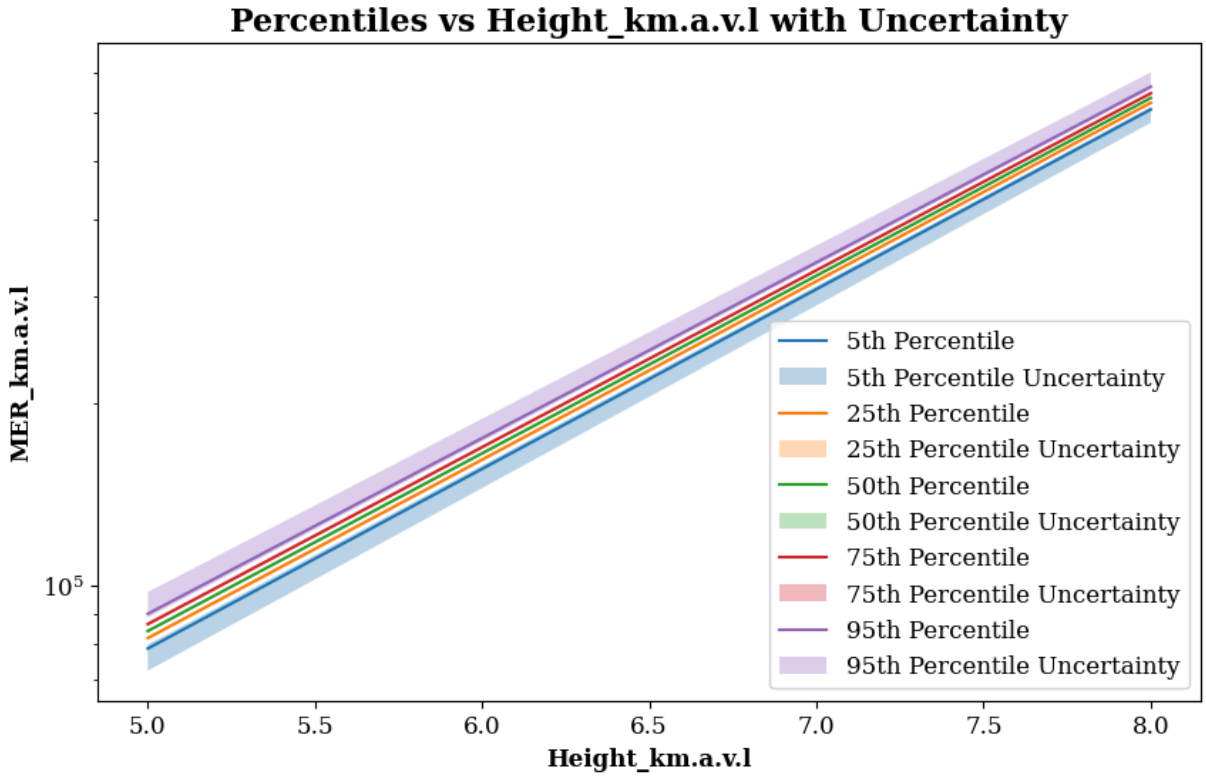
# Visualize percentiles with uncertainty
predictor.plot_percentiles_with_uncertainty(Observed_plume_height)

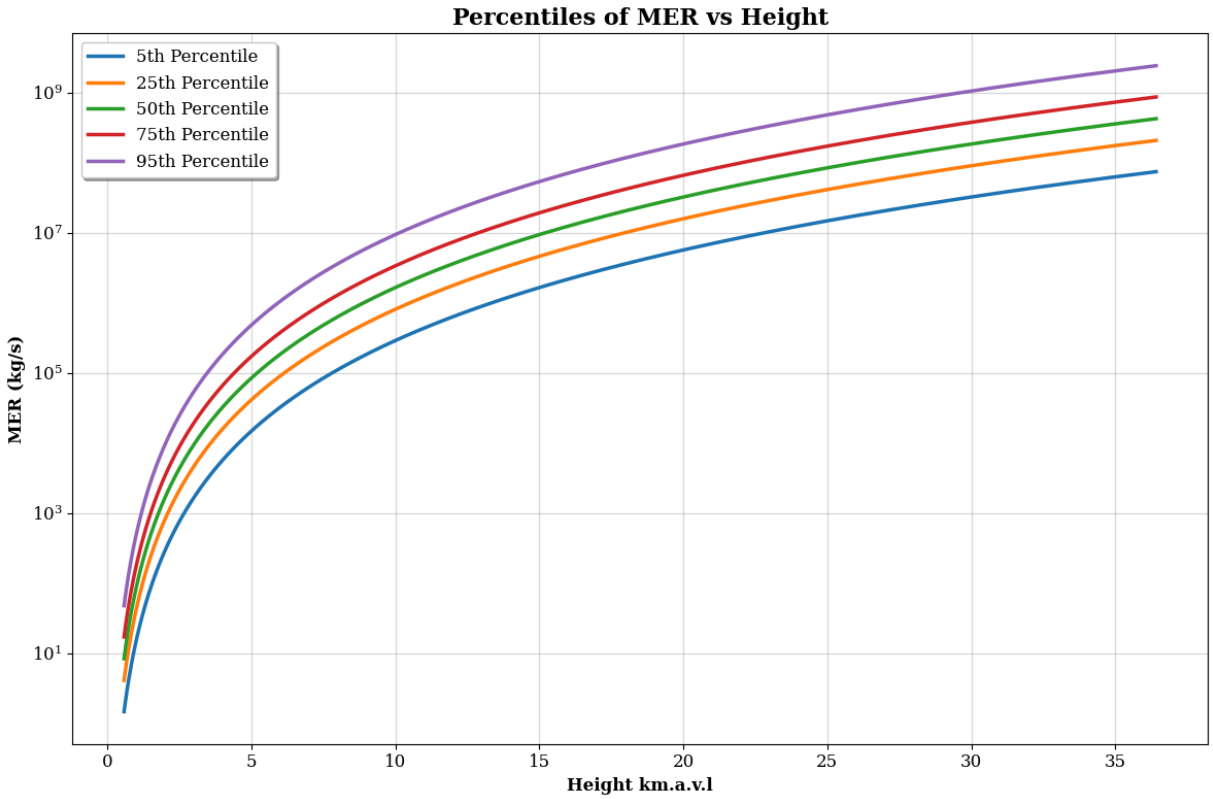
predictor.plot_percentiles_vs_mer()

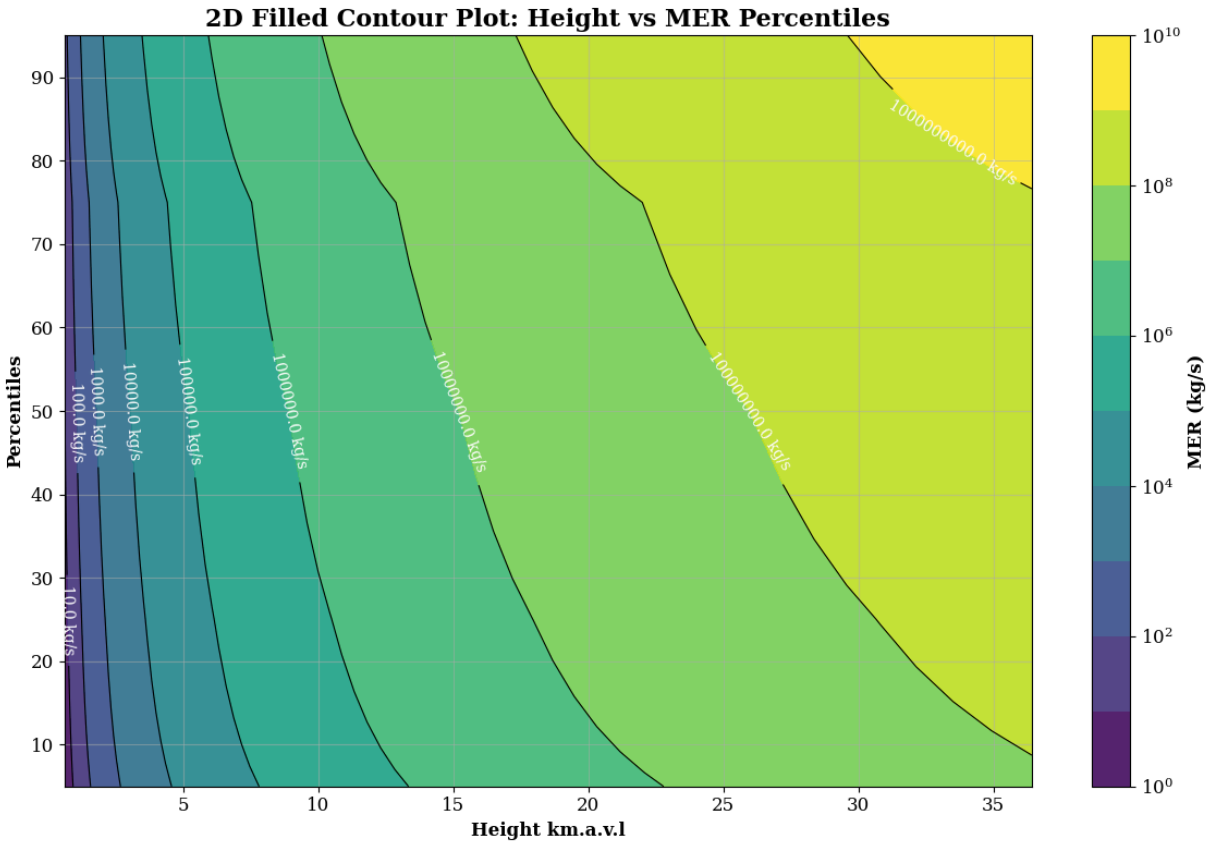
predictions
```

```
Predictions with uncertainty saved to predicted_mer_fromheight.csv
```




Percentiles vs Height_km.a.v.l with Uncertainty
MER_km.a.v.l
10^5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
Height_km.a.v.l
5th Percentile
5th Percentile Uncertainty
25th Percentile
25th Percentile Uncertainty
50th Percentile
50th Percentile Uncertainty
75th Percentile
75th Percentile Uncertainty
95th Percentile
95th Percentile Uncertainty


Percentiles of MER vs Height
5th Percentile
25th Percentile
50th Percentile
75th Percentile
95th Percentile
10^9
10^7
10^5
10^3
10^1
MER (kg/s)
0
5
10
15
20
25
30
35
Height km.a.v.l



Out[47]:

	Height_km.a.v.l	MERPercentile_5	Uncertainty_Lower_5	Uncertainty_Upper_5	MERPercen
0	5	78818.399256	72488.975652	80006.726657	82014.
1	8	607683.878811	577891.961330	613530.493349	623341.

In [48]:

```
from ParticleSize_MER import Predict_ASH_BELOW_63_Micron

from vei import data_loader

#mastin_a=data_loader.load_Mastin_a(as_geodataframe=True)

psize_model=Predict_ASH_BELOW_63_Micron(mastin_a)

psize_model.set_xvar('MER_kg/s')
psize_model.set_yvar('MASS_FRACTION_ASH_BELOW_63_micron')


psize_model.calculate_percentiles_with_uncertainty(mer_list)
psize_model.predict_with_uncertainty(mer_list)

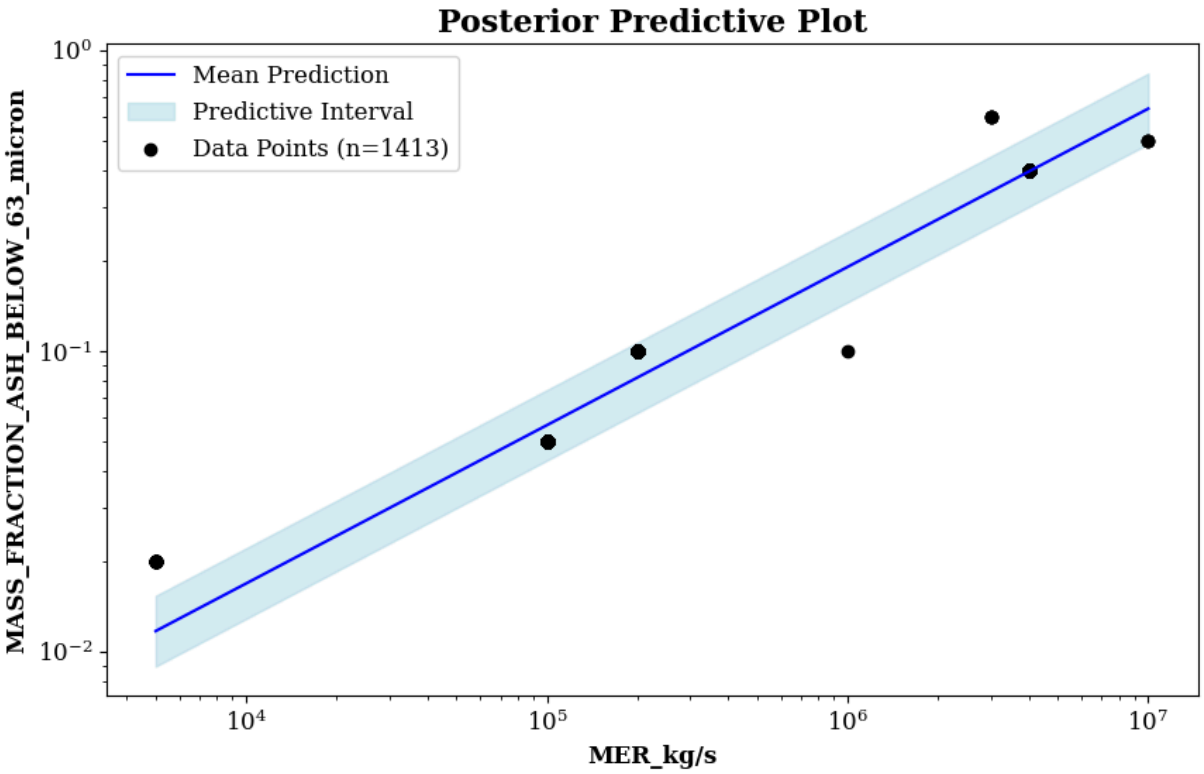
psize_model.plot_posterior_predictive()

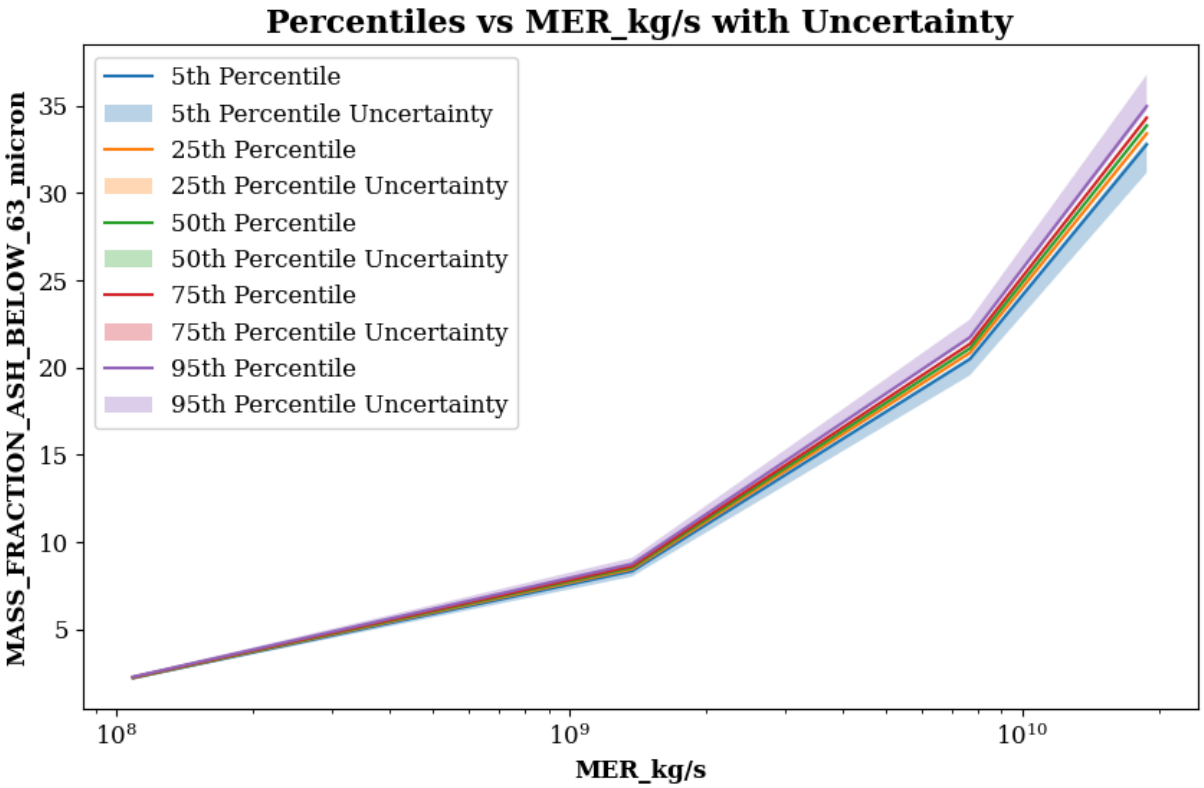

# Visualize percentiles with uncertainty
psize_model.plot_percentiles_with_uncertainty(mer_list)

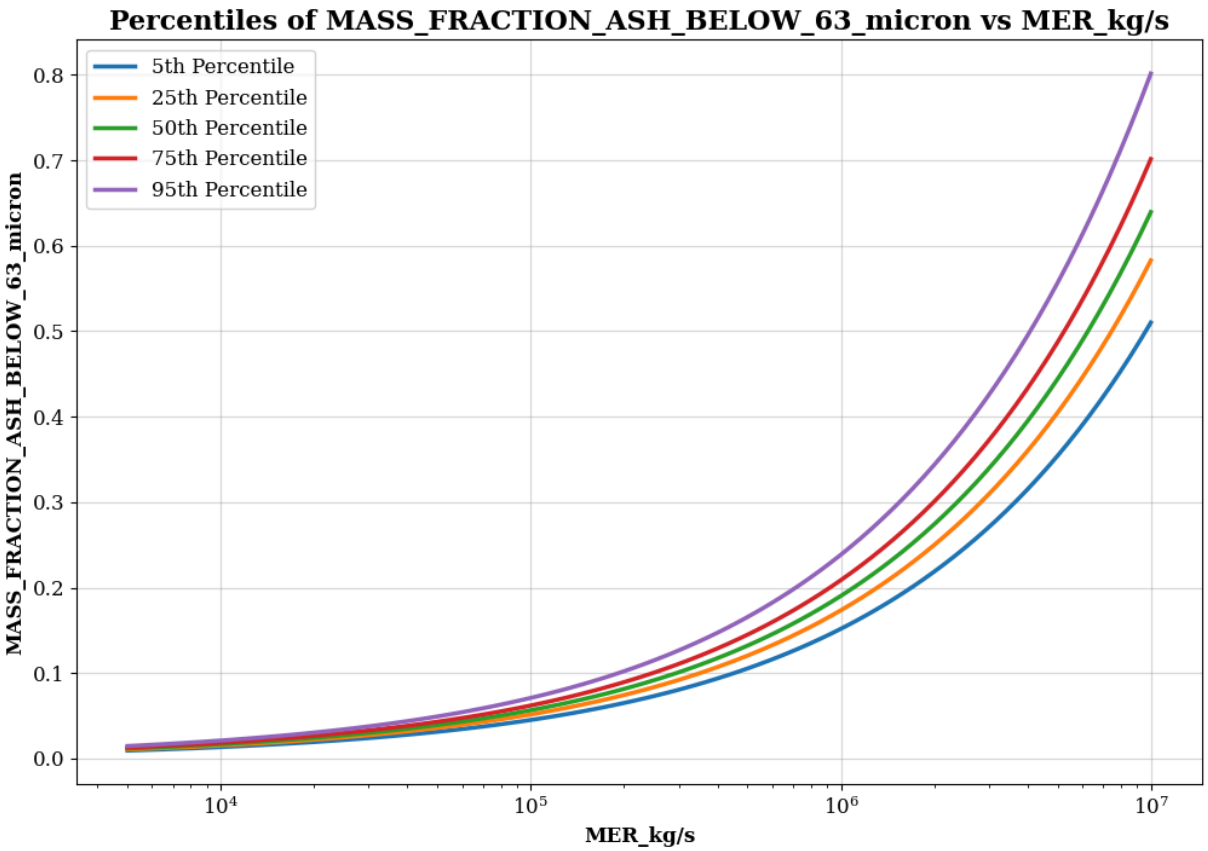
psize_model.plot_percentiles()
```

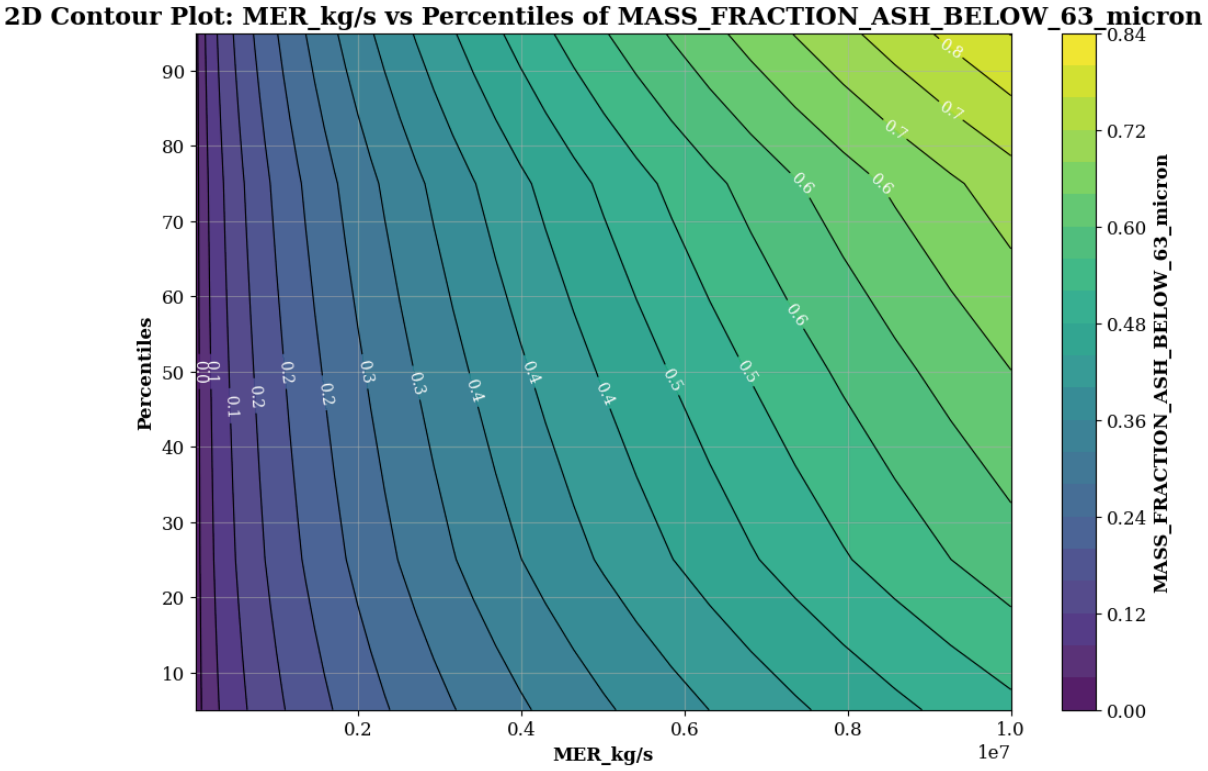
```
psize_predictions=psize_model.predict_with_uncertainty(mer_list)
```

```
Model updated with xvar='MER_kg/s' and yvar='MER_kg/s'.
Model updated with xvar='MER_kg/s' and yvar='MASS_FRACTION_ASH_BELOW_63_micron'.
```









In [49]:
```
psize_predictions
```

Out[49]:

	MER_kg/s	MASS_FRACTION_ASH_BELOW_63_micron_Percentile_5	Uncertainty_Lower_5
0	1.089304e+08	2.210497	2.154427
1	1.376456e+09	8.338536	8.026694
2	7.666528e+09	20.485053	19.555345
3	1.882030e+10	32.774524	31.153115

In [50]:

```
columns_to_assign=list(psize_predictions.columns)
columns_to_assign=columns_to_assign[1:]

# Assuming `columns_to_assign` contains column names and `heights` and `psize_predi

heights.loc[:, columns_to_assign] = psize_predictions[columns_to_assign]


heights.to_csv('Final_prediction_results.csv')
```

In [51]:

```
psize_predictions
```

Out[51]:

	MER_kg/s	MASS_FRACTION_ASH_BELOW_63_micron_Percentile_5	Uncertainty_Lower_5
0	1.089304e+08	2.210497	2.154427
1	1.376456e+09	8.338536	8.026694
2	7.666528e+09	20.485053	19.555345
3	1.882030e+10	32.774524	31.153115